



A FINVERGE PRODUCT

Third-Party Integration & Handshaking Specification

Lane B: Core Banking (CBS) & Loan Origination (LOS) Feeds

Lane A: Real-Time Decisioning for UPI, IMPS & Cards

Prepared for third-party integration teams: core banking system (CBS) vendor, loan origination system (LOS) vendor, payment switch / UPI-IMPS provider, and card network / issuer processor.

Document classification: Confidential — shared under integration NDA

Version 1.1 · 17 August 2026

Issued by: Fraud360 Platform Engineering

Document Control

Field	Value
Document title	Fraud360 Third-Party Integration & Handshaking Specification — Lane A (UPI/IMPS/Cards) & Lane B (CBS/LOS)
Version	1.1
Status	Issued for integration
Audience	Bank/NBFC-appointed integration teams for: payment switch (UPI/IMPS), card network / issuer processor, core banking system (CBS), and loan origination system (LOS)
Distribution	Shared per-engagement, under NDA. Lane A and Lane B recipients receive the same document; each section states which lane it applies to
Owner	Fraud360 Platform Engineering
Related documents	Fraud360 Functional Specification (FSD), High-Level Design (HLD) — available to the bank's own project team on request; not distributed to vendors

How to read this document

Every request/response shown here is the actual contract implemented by the running platform — field names, validation rules, status codes and error shapes are taken directly from the service source, not drafted separately. If your test environment returns something different from what is documented here, treat that as a defect to report, not a documentation gap to work around.

Contents

1. Purpose and audience

2. Architecture overview — two lanes, two integration surfaces

3. Common conventions (apply to both lanes)

- 3.1 Network endpoints and environments
- 3.2 Authentication — machine credentials
- 3.3 Canonical data conventions — money, time, identifiers
- 3.4 Batch semantics — partial acceptance and deduplication
- 3.5 Error response format

4. Part A — Lane A: real-time decisioning for UPI, IMPS and Cards

- 4.1 What this integration is for
- 4.2 POST /decide — request and response contract
- 4.3 Rail capabilities, latency budgets and fail policy
- 4.4 Shadow mode
- 4.5 Worked examples
- 4.6 Response handling reference
- 4.7 Complementary feed: settled-transaction intake
- 4.8 ISO 8583 (Preview) — for switches that cannot originate JSON

5. Part B — Lane B: CBS and LOS feed handshaking

- 5.1 What this integration is for
- 5.2 POST /ingest/{tenant_id}/cbs-events — request and response contract
- 5.3 Event kinds and field reference
- 5.4 Coverage and status visibility
- 5.5 Worked example

6. Onboarding checklist

Appendix A — Lane A payment-rail field reference (UPI / IMPS / Card)

Appendix B — Error code reference

Appendix C — Contacts and change control

1. Purpose and Audience

Fraud360 is the bank's early-warning and fraud-risk-management platform. It ingests transaction and account-conduct data from the bank's own systems, screens it against a rule catalogue aligned to the RBI Master Directions on Fraud Risk Management, and returns either a real-time decision (Lane A) or a near-real-time alert (Lane B).

This document specifies exactly what a third-party system must send, in what shape, and what it will receive back, to integrate with Fraud360 on two independent surfaces:

- **Lane B handshaking** — the bank's Core Banking System (CBS) and Loan Origination System (LOS) deliver account-conduct and loan-lifecycle events to Fraud360 as a batch/event feed. Scoring happens after ingestion, with full account context.
- **Lane A handshaking** — the bank's payment switch (for UPI and IMPS) and card network / issuer processor call Fraud360 synchronously, inside the payment authorisation window, and receive a decision before the payment is completed.

Both integrations are covered in this single document because they share the same authentication mechanism, data conventions and error format (§3). Sections 4 and 5 are independent of each other — a CBS/LOS integrator only needs §3 and §5; a payment-switch or card-processor integrator only needs §3 and §4.

Scope note

This specification covers UPI, IMPS and Card on Lane A, per the current integration request. NEFT and RTGS are supported by the platform but are scored on Lane B (near-real-time, after ingestion) because their settlement window is wide enough for full-context screening — they are out of scope for the real-time /decide contract in §4 and are not discussed further here.

2. Architecture Overview — Two Lanes, Two Integration Surfaces

Fraud360 screens transactions on two lanes that trade off latency against context. A third-party integrator only needs to know which lane its payload belongs to and what that lane promises.

Lane A — inline, synchronous, in the payment window

UPI, IMPS and Card authorisations are decided while the payment is still in flight. The calling system (payment switch or card processor) sends the payment to POST /decide and receives an allow/decline/challenge answer inside a fixed latency budget (80–150 ms depending on rail — see §4.3). Because this call sits inside the customer's payment, it is deliberately the smallest, fastest surface Fraud360 exposes: no aggregate query, no lookup outside an in-memory counter store, and the decision log is written after the answer is returned so persistence never adds to the customer's wait.

Lane B — near-real-time / batch, after ingestion

CBS and LOS events are not time-critical in the same way — a loan disbursement or a repayment posting does not need a same-millisecond answer. These are delivered as batches to POST /ingest/{tenant_id}/cbs-events, queued, and scored shortly afterward with the full account history and network context available. Findings surface as alerts in the Fraud360 console and (where configured) as outbound notifications — not as a synchronous response to the CBS/LOS system.

Why the surfaces are kept separate

The two lanes are separate deployable services with different availability classes: if the real-time decision path is down, payments fail; if the batch/event path is briefly delayed, a dashboard is a few minutes stale. Those cannot share a scaling policy or an SLA, so they are never merged — and a CBS/LOS integration will never be asked to meet a payment-authorisation latency budget, nor will a payment switch ever be routed through the batch queue.

One canonical fact, from either lane

Every payment rail and every CBS/LOS event is normalised at the edge into one canonical shape before scoring ever sees it. This is why the field-level contracts in §4 and §5 look similar in structure (an amount, a timestamp, an account, a counterparty) even though the source systems are completely different — the rule catalogue is written once, against one shape, not once per source system.

3. Common Conventions (Apply to Both Lanes)

3.1 Network Endpoints and Environments

Fraud360 exposes three distinct network endpoints. They are on different hosts by design (see box below), not a documentation inconsistency — provision your integration to call each at the base URL Fraud360 issues to you at onboarding.

Capability	Called by	Path (fixed)	Base URL
Token issuance	All integrators	POST /machine/token	Identity service — issued at onboarding
Batch/event intake (payment + CBS/LOS)	CBS, LOS, and payment switch (for the settled-record feed, §4.7)	POST /api/ingest/{tenant_id}/transactions POST /api/ingest/{tenant_id}/cbs-events	API Gateway — issued at onboarding
Real-time decision	Payment switch (UPI/IMPS), card processor	POST /decide?tenant_id={tenant_id}	Decision service — issued at onboarding, separate host from the gateway

Every sample request in this document uses illustrative placeholder hostnames — your bank's actual base URLs are issued per environment (UAT and Production) at onboarding, and are not derivable from this document:

```
Identity service (token issuance) -> https://idp.fraud360.examplebank.co.in
API Gateway (batch/event intake) -> https://api.fraud360.examplebank.co.in
Decision service (real-time /decide) -> https://decision.fraud360.examplebank.co.in
```

Why the real-time endpoint is a separate host

The /decide endpoint is not routed through the API gateway that fronts every other Fraud360 API. It is reached directly, on its own host, because the availability class of the payment-authorisation path must not be coupled to the console/reporting edge — a slow or degraded gateway must never be able to add latency to a payment. Call /decide at the dedicated decision-service URL you are issued, not at the gateway URL used for batch intake.

3.2 Authentication — Machine Credentials

Both lanes authenticate with the same mechanism: a machine credential, issued by your bank's Fraud360 tenant administrator, exchanged for a short-lived bearer token. This is a distinct credential class from a human console

login — it has no interactive login step and no MFA challenge, because a payment switch or a nightly CBS batch job cannot complete one. In its place, Fraud360 applies compensating controls (below).

Credential scopes

Scope	Grants	Typical holder
ingest	Submit payment-rail transactions and/or CBS/LOS events (§4.7, §5.2)	CBS, LOS, payment switch (settled-record feed)
decide	Call the real-time decision endpoint (§4.2)	Payment switch, card processor
ingest+decide	Both of the above	Payment switch handling both the real-time call and the settled-record feed

Token exchange

OAuth2 client-credentials grant, spelled the way integration teams generally expect it:

REQUEST

```
POST https://idp.fraud360.examplebank.co.in/machine/token
Content-Type: application/json

{
  "grant_type": "client_credentials",
  "client_id": "svc_3063a96d7d0b17e1",
  "client_secret": "<issued once at credential creation, never retrievable again>"
}
```

RESPONSE — 200

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..",
  "token_type": "Bearer",
  "expires_in": 1800,
  "scope": "ingest+decide",
  "tenant_id": "b3b90261-9663-46c3-b31f-e8afc4a65442"
}
```

Tokens expire after **1800 seconds (30 minutes)** by default. Re-authenticate on expiry rather than caching indefinitely; do not request a fresh token per payment — once per token lifetime is the intended pattern. Present the token on every subsequent call as Authorization: Bearer <token>.

Compensating controls (in place of MFA)

- **Certificate binding** (optional, recommended) — bind the credential to a client certificate, presented either via mutual TLS or forwarded by your edge as the X-Client-Cert-Thumbprint header. A mismatch is refused.
- **IP allow-listing** — restrict the credential to the calling system's known egress addresses.
- **Expiry and rotation** — every credential has an expiry (default 365 days); rotate before it lapses. A rotated or revoked credential is refused with a reason, not a generic failure.
- **Full audit trail** — every token issuance and every refusal (wrong secret, expired, off-list address, certificate mismatch) is recorded against the credential, tenant, and calling address.

Enumeration resistance

An unknown client_id and a correct client_id with the wrong secret return the identical 401 invalid_client response. This is deliberate — it prevents a caller from using the token endpoint to discover which client ids exist. Do not attempt to distinguish these two cases from the response; the resolution for both is the same (verify your issued credential with your Fraud360 onboarding contact).

3.3 Canonical Data Conventions

Money — always integer paise

Every amount field in every Fraud360 API is an integer number of paise (amount_paise), never a decimal rupee value and never a float. ₹1,234.55 is sent as 123455. Where a field named plainly amount is accepted (see rail-specific tables in §4.7 and §5.3), Fraud360 parses it as an exact decimal and converts internally — sub-paisa precision (e.g. an amount implying a fraction of a paisa) and negative amounts are both rejected outright.

Time — timestamps must carry a timezone

Every timestamp must be ISO-8601 with an explicit UTC offset, e.g. 2026-08-05T14:32:10+05:30. A timestamp with no offset is **rejected**, not assumed to be IST. This is deliberate: guessing wrongly about a timezone silently shifts every velocity/time-window rule by up to 5.5 hours, and a wrong-but-plausible answer is worse than an explicit failure your integration will actually notice and fix.

Identifiers and idempotency

Fraud360 deduplicates on *your* system's own identifier for each record — the payment rail's transaction id for a payment (§4.7), and the CBS/LOS system's own event id for a Lane B event (§5.2). Re-sending a record you have already sent (a retried file, a re-run nightly extract) is treated as a normal, expected duplicate, reported back to you, and costs nothing — it is never double-counted or double-scored.

3.4 Batch Semantics

Both intake endpoints (/ingest/{tenant_id}/transactions and /ingest/{tenant_id}/cbs-events) share one contract:

- **202 Accepted, not 200/201** — the platform has taken responsibility for the batch but has not yet scored it. A 202 response means "queued", not "screened".
- **Partial acceptance** — a batch is never rejected wholesale because one row is malformed. Every row is adapted and validated independently; a bad row is reported with its index and reason while every good row in the same batch proceeds.
- **Duplicates are not errors** — a row that dedupes against one already received is counted separately from rejected rows and does not fail the batch.
- **Batch size limit** — maximum 5,000 rows per request (transactions or CBS/LOS events). Split larger deliveries into multiple requests.

RESPONSE SHAPE — BOTH ENDPOINTS

```
HTTP/1.1 202 Accepted

{
  "accepted": 4991,
  "duplicates": 6,
  "rejected": 3,
  "errors": [
    { "index": 118, "rail": "UPI", "error": "missing after mapping: amount_paise" },
    { "index": 2044, "error": "timestamp has no timezone. Send an offset (e.g.
+05:30) ..." }
  ]
}
```

```
// up to 50 error entries are returned per response; the batch itself may reject
more
],
"batch_id": "f0c1a3d2-7b9e-4a11-8f3d-1c2b6e9a0d7f"
}
```

Design intent

Rejecting 5,000 good rows because one has a malformed timestamp is how an integration ends up building its own retry queue and resending everything, doubling load for no benefit. Fix and resend only the rows named in errors — everything else in the batch has already been accepted.

3.5 Error Response Format

Every 4xx/5xx response, on every Fraud360 endpoint, uses one JSON envelope:

```
{
  "error": {
    "code": "rail_not_inline",
    "message": "Rail 'NEFT' is configured for the near-real-time lane. It is scored
               with full context after ingestion, not in the payment window."
  }
}
```

Integrate against `error.code` (stable, machine-readable) for branching logic; show or log `error.message` (human-readable, may change wording) for diagnostics. A full code reference for both lanes is in Appendix B.

4. Part A — Lane A: Real-Time Decisioning for UPI, IMPS and Cards

Applies to

Payment switch integration team (UPI, IMPS) and card network / issuer processor integration team. Requires a machine credential scoped decide (§3.2).

4.1 What This Integration Is For

Your system calls POST /decide once per UPI, IMPS or Card authorisation, synchronously, before the payment is completed. Fraud360 evaluates the payment against the bank's configured rule catalogue and returns an action your system must act on. There is no separate "submit" step for this lane — the call to /decide *is* the submission, and Fraud360 records the decision after responding so persistence never adds to your wait.

4.2 POST /decide — Request and Response Contract

Called directly at the decision-service base URL (§3.1) — never through the API gateway. tenant_id is a query parameter, not part of the path.

Request fields

Field	Type	Required	Notes
txn_ref	string ≤64	Yes	Your own reference for this payment. Echoed nowhere in the response — kept only in Fraud360's decision log for later lookup.
rail	string ≤8	Yes	One of UPI, IMPS, CARD (for this integration). Determines which policy and latency budget apply.
debtor_account	string ≤64	Yes	The paying side.
creditor_account	string ≤64	No (default "")	The receiving side.
amount_paise	integer ≥0	Yes	Integer paise — see §3.3. Never a decimal rupee value.
channel	string ≤16	No	e.g. mobile, netbanking, pos. Free text, used only as a detection signal.
device_id	string ≤64	No	Device fingerprint if your channel captures one.
signals	object (string→number)	No (default {})	Observations your channel already holds at authorisation time — see note below.

The signals object — send what you already know

Fraud360 does not re-derive per-session observations (e.g. "is this a new device paying a new beneficiary") inside the 80–150 ms budget, because a freshly rebuilt context has no memory of prior sessions and would misreport every payment as novel. Your channel is the better-informed party for this one figure — send it as signals.risk_signals (a small integer count of risk indicators your channel has already flagged, e.g. new device + new payee + odd hour = 3). If omitted, the rule that depends on it is reported in the response's skipped array with its reason, not silently treated as zero.

Response fields

Field	Type	Meaning
action	string	What your system must act on: allow / decline / challenge. This already accounts for shadow mode (below) — always act on this field, never on would_be.

Field	Type	Meaning
would_be	string	What enforcement would have returned, whether or not shadow mode suppressed it. For reporting only — never act on this field.
outcome	string	clean (screened, nothing matched) / matched / not_screened / store_unavailable / budget_exceeded. See §4.6.
enforced	boolean	true if action reflects real enforcement; false if the rail is in shadow mode (action is forced to allow).
shadow	boolean	Whether this rail is currently in shadow mode.
score	number	Aggregate rule score.
severity	string	low / medium / high / critical, per the tenant's configured bands.
matched	array	Each matched rule: rule_id, family, sub_rule_ref, observed, threshold, observed_unit, reason.
skipped	array	Each rule that could not be measured for this payment, with a reason. Not a failure — an honest "not evaluated", never counted as a pass.
took_ms	number	How long Fraud360 took to answer.
budget_ms	integer	The latency budget configured for this rail (§4.3).
policy_version	string	Which configuration version produced this decision — needed to interpret a decision correctly months later, after thresholds have moved.
reference	string	Fraud360's own reference for this decision, for later lookup/dispute.

4.3 Rail Capabilities, Latency Budgets and Fail Policy

An action your rail cannot actually execute is worse than no action at all — a challenge returned to a rail with no step-up path is simply ignored by the switch while the record shows a control that fired. Fraud360 enforces this by construction: a rail is only ever configured with actions it can carry.

Rail	Actions it can carry	Why	Latency budget (starter policy)	Fail policy
UPI	decline only	The PIN is captured at the remitter PSP before the request reaches the bank — the customer has already authenticated, so the only possible answers are approve or decline. A returned challenge would be ignored.	80 ms	open*
IMPS	decline only	Authenticated upstream in the initiating channel; there is no step-up the beneficiary-side decision can insert.	150 ms	open*
CARD	challenge, decline	3-D Secure makes a challenge a first-class protocol outcome — the one rail in this integration where a genuine step-up is available inside the authorisation.	100 ms	open*

* fail policy is configured per rail by the bank, not defaulted by Fraud360 — see box below.

What "fail open" means, and who decides it

If Fraud360 is unreachable or does not answer inside your timeout, your system applies its own timeout and the rail's fail policy decides what happens next: fail-open lets the payment proceed unscreened, fail-closed declines it. Fraud360 never defaults this silently — every inline rail has an explicit fail policy set by the bank. Confirm the fail

policy for each rail with your Fraud360 onboarding contact before go-live; it determines what your integration should do on a timeout, and it may differ from the starter values shown above once the bank tunes its own policy.

Requesting an action a rail cannot carry is a configuration error the bank makes when setting up the policy, not something your integration triggers — but if you ever see an action value your rail cannot execute, treat it as a defect to report immediately rather than something to work around.

4.4 Shadow Mode

Every rail starts in shadow mode (`shadow: true`). Fraud360 computes the real decision, records what it *would* have returned in `would_be`, and always returns `action: "allow"` while shadow is on — so the bank can run Fraud360 against live traffic and measure false-positive cost before a single real payment is ever declined. Enforcement is turned on per rail by the bank as a deliberate configuration change with its own approval trail; it is not something your integration requests or controls. Your integration code should be written once, against action, and does not need to change when the bank later turns enforcement on.

4.5 Worked Examples

All three payloads below are the same endpoint and the same contract; only the rail, the budget and the outcome differ. Bearer tokens are abbreviated; nothing else is edited.

Example 1 — UPI, clean (allowed)

REQUEST

```
POST https://decision.fraud360.examplebank.co.in/decide?tenant_id=b3b90261-9663-46c3-b31f-e8afc4a65442
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
Content-Type: application/json

{
  "txn_ref": "UPI-2026-0817-000191",
  "rail": "UPI",
  "debtor_account": "AC-HLD-CALM",
  "creditor_account": "AC-BEN-77120",
  "amount_paise": 250000,
  "channel": "mobile",
  "device_id": "d41d8cd98f00b204",
  "signals": { "risk_signals": 0 }
}
```

RESPONSE — 200, 4.2 MS

```
{
  "reference": "72492b3d91e24a4b",
  "action": "allow",
  "would_be": "allow",
  "outcome": "clean",
  "score": 0.0,
  "severity": "low",
  "enforced": false,
  "shadow": true,
  "matched": [],
  "skipped": [ /* rules not measurable from this payload alone, each with its reason
  */ ],
  "took_ms": 4.16,
  "budget_ms": 80,
  "policy_version": "1.0.0"
}
```

```
}

```

Example 2 — UPI, matched (would be declined)

REQUEST

```
POST https://decision.fraud360.examplebank.co.in/decide?tenant_id=b3b90261-9663-46c3-b31f-e8afc4a65442
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
Content-Type: application/json

{
  "txn_ref": "UPI-2026-0817-000192",
  "rail": "UPI",
  "debtor_account": "AC-HLD-HOT",
  "creditor_account": "AC-BEN-99001",
  "amount_paise": 500000,
  "channel": "mobile",
  "device_id": "9e107d9d372bb682",
  "signals": { "risk_signals": 5 }
}
```

RESPONSE — 200, 1.7 MS

```
{
  "reference": "77de6e3a0eba4fb6",
  "action": "allow",           // SHADOW: the payment proceeds
  "would_be": "decline",      // enforcing, it would not have
  "outcome": "matched",
  "score": 600.0,
  "severity": "critical",
  "enforced": false,
  "shadow": true,
  "matched": [
    { "rule_id": "CHN-01", "family": "CHN", "observed": 5.0, "threshold": 3.0,
      "observed_unit": "risk_signals",
      "reason": "New device + new beneficiary + high value in one session" },
    { "rule_id": "LAY-02", "family": "LAY", "observed": 55.0, "threshold": 40.0,
      "observed_unit": "distinct_counterparties",
      "reason": "40+ distinct counterparties in window (fan-in/fan-out hub)" }
    // ... 3 more matches, each with the same shape
  ],
  "skipped": [ /* 20 entries, each with its reason */ ],
  "took_ms": 1.663,
  "budget_ms": 80,
  "policy_version": "1.0.0"
}
```

Example 3 — IMPS, matched (would be declined)

Identical contract to UPI; only the budget differs, and it comes from the rail's own configured policy — IMPS is given 150 ms against UPI's 80 ms because their settlement windows differ, not because the code does.

REQUEST

```
POST https://decision.fraud360.examplebank.co.in/decide?tenant_id=b3b90261-9663-46c3-b31f-e8afc4a65442
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
Content-Type: application/json
```

```
{
  "txn_ref": "IMPS-2026-0817-004417",
  "rail": "IMPS",
  "debtor_account": "AC-HLD-HOT",
  "creditor_account": "AC-BEN-99002",
  "amount_paise": 4500000,
  "channel": "netbanking",
  "device_id": "9e107d9d372bb682",
  "signals": { "risk_signals": 4 }
}
```

RESPONSE — 200, 1.5 MS

```
{
  "reference": "73c5b65625cb4a7d",
  "action": "allow",
  "would_be": "decline",
  "outcome": "matched",
  "score": 600.0,
  "severity": "critical",
  "enforced": false,
  "shadow": true,
  "matched": [
    { "rule_id": "VEL-01", "family": "VEL", "observed": 450.0, "threshold": 5.0,
      "observed_unit": "value_vs_baseline",
      "reason": "Transaction value 450x this account's own 30-day baseline" }
  ],
  "skipped": [ /* ... */ ],
  "took_ms": 1.476,
  "budget_ms": 150,           // IMPS, not UPI's 80
  "policy_version": "1.0.0"
}
```

Example 4 — Card, challenge via 3-D Secure

Card is the one rail in this integration where action can be challenge — your processor should route this to a 3-D Secure step-up rather than declining outright.

REQUEST

```
POST https://decision.fraud360.examplebank.co.in/decide?tenant_id=b3b90261-9663-46c3-b31f-e8afc4a65442
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
Content-Type: application/json

{
  "txn_ref": "CARD-2026-0817-118820",
  "rail": "CARD",
  "debtor_account": "TOK-4f9e2c7a1b6d",
  "creditor_account": "MERCH-88213-ELECTRONICS",
  "amount_paise": 1899900,
  "channel": "pos",
  "device_id": "",
  "signals": { "risk_signals": 2 }
}
```

RESPONSE — 200, 2.9 MS

```
{
  "reference": "9a1e5b3c7d2f4088",
```

```

"action": "challenge",           // enforced: this rail is no longer in shadow
"would_be": "challenge",
"outcome": "matched",
"score": 210.0,
"severity": "medium",
"enforced": true,
"shadow": false,
"matched": [
  { "rule_id": "CHN-02", "family": "CHN", "observed": 2.0, "threshold": 2.0,
    "observed_unit": "risk_signals",
    "reason": "Card-not-typical merchant category at this value band" }
],
"skipped": [ /* ... */ ],
"took_ms": 2.94,
"budget_ms": 100,
"policy_version": "1.0.0"
}

```

Example 5 — A rail this lane does not handle

NEFT and RTGS are scored on Lane B with full account and network context after ingestion. Calling /decide for one is a configuration error and is refused, not answered with allow — an allow here would be indistinguishable from a payment that actually was screened and passed.

RESPONSE — 409

```

{
  "error": {
    "code": "rail_not_inline",
    "message": "Rail 'NEFT' is configured for the near-real-time lane. It is scored
               with full context after ingestion, not in the payment window."
  }
}

```

4.6 Response Handling Reference

Status	Code	When	What your system should do
200	—	A decision was reached	Act on action. Never act on would_be — it is for reporting only.
401	invalid_client	Unknown client id or wrong secret on token exchange	Do not retry with the same credential; alert your integration team.
403	credential_unusable	Credential expired or revoked, with the reason	Rotate the credential — expiry is deliberate, not a fault.
403	ip_not_allowed / certificate_mismatch	Calling address off the allow-list, or certificate does not match the binding	Treat as a security event, not a configuration bug.
403	tenant_suspended / tenant_offboarded	The bank's Fraud360 service is stopped	Stop calling this endpoint; contact your Fraud360 onboarding lead.
403	(scope refusal)	An ingest-only credential was used to call /decide	Request a credential scoped decide or ingest+decide.
409	rail_not_inline	The rail is configured for Lane B, not real-time decisioning	Do not call /decide for this rail — it is screened on Lane B instead.

Status	Code	When	What your system should do
409	no_rail_policy	No inline policy exists yet for this rail	A configuration gap on the bank's side. Escalate rather than assume it is safe to treat as allow.
Timeout / no response	—	Fraud360 did not answer inside your timeout	Apply the rail's configured fail policy (§4.3) — never assume open or closed.

4.7 Complementary Feed: Settled-Transaction Intake

The real-time call in §4.2 decides the payment; it does not, by itself, give Fraud360 the settled transaction record for ongoing account-conduct scoring, historical reporting, or the near-real-time rules that need full context. Send the same payment — after settlement — to the batch intake endpoint below. This is a separate, additive feed, not a replacement for §4.2.

Attribute	Detail
Endpoint	POST <code>https://api.fraud360.examplebank.co.in/api/ingest/{tenant_id}/transactions</code>
Auth	Machine credential scoped ingest (or ingest+decide)
Batch semantics	As in §3.4 — 202 Accepted, partial acceptance, max 5,000 rows/request, dedupes on the rail's own transaction id
Delivery options	REST batch (below) or file/SFTP drop — CSV, JSON-lines, or ISO 20022 (pacs.008 / pain.001) XML. See Appendix A for the file convention.

Sample batch — one row per rail

REQUEST

```
POST https://api.fraud360.examplebank.co.in/api/ingest/b3b90261-9663-46c3-b31f-e8afc4a65442/transactions
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6I..
Content-Type: application/json

{
  "source": "live",
  "transactions": [
    { "rail": "UPI", "payload": {
      "upiTransactionId": "UPI-2026-0817-000191",
      "txnTimestamp": "2026-08-17T09:12:04+05:30",
      "amountRupees": "2500.00",
      "payerAccount": "AC-HLD-CALM", "payeeVpa": "merchant@examplebank",
      "channel": "mobile", "status": "settled" } },
    { "rail": "IMPS", "payload": {
      "rrn": "IMPS-2026-0817-004417",
      "txnDate": "2026-08-17T09:14:51+05:30",
      "amount": "45000.00",
      "remitterAccount": "AC-HLD-HOT", "beneficiaryAccount": "AC-BEN-99002",
      "channel": "netbanking" } },
    { "rail": "CARD", "payload": {
      "authCode": "CARD-2026-0817-118820",
      "authTime": "2026-08-17T09:20:33+05:30",
      "authAmount": "18999.00",
      "pan_token": "TOK-4f9e2c7a1b6d", "merchantId": "MERCH-88213-ELECTRONICS",
      "channel": "pos" } }
  ]
}
```

```
}
```

RESPONSE — 202

```
{
  "accepted": 3,
  "duplicates": 0,
  "rejected": 0,
  "errors": [],
  "batch_id": "1c9b2a44-6e7d-4f10-9a3c-8d5e2b7f0c11"
}
```

Field-by-field accepted key names for each rail (UPI, IMPS, Card), including every alias Fraud360 recognises, are in Appendix A.

4.8 ISO 8583 (Preview) — for Switches That Cannot Originate JSON

Status: Preview, not for production integration yet

This section describes a second Lane A front door for card/EFT switches that have no practical way to call the JSON /decide contract in §4.2. The core protocol adapter — message codec, field mapping, and connection handling — is built and passes its full test suite, including live end-to-end tests against the real decision engine using synthesised authorisation traffic. It has not been connected to any live switch. Two things must be confirmed with your integration team before this becomes a production contract: your switch's exact ISO 8583 dialect and private-field usage (§4.8.4), and whether Fraud360 terminating your raw PAN brings it into your PCI-DSS cardholder-data scope (§4.8.5). Treat everything below as a preview of the shape this integration will take, not a final specification — do not build production code against it yet.

4.8.1 Why a Second Front Door

Card/EFT switches are frequently on-premises, vendor-licensed systems with no REST layer available without a vendor change request. Rather than asking every such vendor to build a JSON translation layer of their own, Fraud360 provides a native ISO 8583 listener that decodes an authorisation message and calls the /decide contract in §4.2 internally — same rule catalogue, same policy, same decision log. Two front doors, one decision engine: nothing about how a payment is scored differs based on which protocol carried it in.

4.8.2 Transport and Message Types

Attribute	Detail
Transport	Persistent TCP connection, one per switch. Base host/port issued per bank at onboarding — separate from the base URLs in §3.1.
Framing	2-byte big-endian length header, followed by the MTI + bitmap + field data. (A 4-byte header is a documented alternative if your switch requires it — confirm at onboarding.)
Encoding	ASCII only, in this preview. BCD/EBCDIC-encoded fields are not yet supported.
Request MTI	0100 (authorisation request) or 0120 (authorisation request — advice)
Response MTI	0110 or 0130 respectively

4.8.3 Data Element Mapping (Standard Fields Only)

Only the public, standard data elements defined by the ISO 8583:1987 base standard are mapped in this preview. Every switch vendor's private-use fields (DE48, DE62, DE120–127) differ, and none are mapped here until confirmed against your switch's own specification — see §4.8.4.

ISO 8583 DE	Canonical /decide field	Notes
DE2 (PAN)	debtor_account	Masked at the point it first arrives (hash + last 4). The raw PAN is never sent to /decide, logged, or stored.
DE4 (amount)	amount_paise	—
DE7 (transmission date/time)	Used to build the decision timestamp	Carries no year and no timezone offset — both are inferred against a bank-configured local offset, documented at onboarding.
DE11 / DE37 (STAN / RRN)	txn_ref	RRN preferred where present
DE41 (terminal id)	device_id	—
DE42 (card acceptor id)	creditor_account	—
DE39 (response code, response only)	Built from action	00 = allow, 05 = decline, 01 = challenge (referral — the closest analogue ISO 8583 has to a step-up)

4.8.4 What Is Not Yet Confirmed

- **Your switch's exact dialect** — 1987 vs. 1993 vs. 2003, and which card network's private-field conventions apply (Visa Base I, Mastercard, RuPay/NPCI). No two switches implement ISO 8583 identically outside the standard fields above.
- **Framing details** — length-header width and encoding are confirmed per connection, not assumed from this document.
- **Response code table** — DE39 codes are not perfectly standardised across networks; the mapping in §4.8.3 is a starting point, confirmed against your network's own table before go-live.
- **Certification testing** — many card networks and switch vendors require a certification pass before production traffic. Plan for this as a separate step with its own timeline.

4.8.5 Compliance: PCI-DSS Scope

A decision for your compliance team, not an engineering default

An ISO 8583 authorisation message carries the actual PAN. Whether Fraud360 terminating that connection brings it into your PCI-DSS cardholder-data environment depends on where tokenisation happens in your architecture, and must be resolved with your compliance and security teams before any production connection is established. Fraud360's default behaviour — masking the PAN immediately on arrival and never logging, storing, or forwarding it in raw form — is the safe engineering default, not a substitute for that sign-off.

4.8.6 Next Steps

If your switch cannot call the JSON /decide contract in §4.2, contact your Fraud360 onboarding lead to discuss a pilot. Before any production connection, expect: (1) your switch's own ISO 8583 specification document shared



with Fraud360, (2) a compliance decision on PCI-DSS scope, (3) a UAT connection exercised against agreed test cases, and (4) certification testing per your card network's requirements.

5. Part B — Lane B: CBS and LOS Feed Handshaking

Applies to

Core Banking System (CBS) integration team and Loan Origination System (LOS) integration team. Requires a machine credential scoped ingest (§3.2).

5.1 What This Integration Is For

Five of the platform's detection indicators cannot be answered from payment-rail data at any cost, because the fact they turn on never appears on a payment. A loan repayment looks identical on a bank statement whether it came from the borrower's own funds or from credit taken at another bank; a transfer out of a loan account looks the same whether it went to the approved supplier or somewhere else entirely. The distinguishing fact lives only in the CBS or LOS, and until it is sent, the corresponding indicator is honestly reported as unmeasurable rather than guessed.

RBI EWS rule family	What it needs from your system	Source
BEH-02	The funding source of a loan repayment — was it the borrower's own money, or credit from elsewhere?	CBS
BEH-03	Whether proceeds of a financed asset's sale were actually routed through the lender	CBS
CBS-01	Where disbursed loan funds were subsequently utilised	CBS
CBS-02	Cash withdrawals against a loan/borrowal account (these never traverse UPI/NEFT/RTGS)	CBS
CBS-03	Large transactions with inter-connected group companies	CBS (reference data)
BR-508 (CTR)	Any cash deposit or withdrawal, on any account — for monthly Cash Transaction Report aggregation under PMLA Rule 3	CBS
LOS-01 / LOS-03	Credit application details — applicant identity and declared income, before sanction	LOS
LOS-02	Collateral valuation — asset type and the valuer who assessed it	LOS

5.2 POST /ingest/{tenant_id}/cbs-events — Request and Response Contract

Attribute	Detail
Endpoint	POST <code>https://api.fraud360.examplebank.co.in/api/ingest/{tenant_id}/cbs-events</code>
Auth	Machine credential scoped ingest
Batch semantics	As in §3.4 — 202 Accepted, partial acceptance, max 5,000 events/request, dedupes on your system's own event id
Delivery mechanism	REST batch only — there is no file/SFTP drop for CBS/LOS events (that mechanism exists only for payment rails, §4.7). A nightly extract should be POSTed as one or more batches.

Request body:

```
{
  "source": "cbs",           // or "live" — see note below
  "events": [
    { "payload": { "kind": "loan_repayment", ... } },
  ],
}
```

```
{ "payload": { "kind": "cash_transaction", ... } }
]
```

The source field

Every other Fraud360 fact table's source distinguishes live traffic from demonstration/synthetic data. CBS/LOS events default to "cbs" for backward compatibility, which reads as unknown provenance — fine for detection scoring, but Cash Transaction Report generation (BR-508) is a regulatory filing and refuses to build from anything other than an explicit "live". If your feed is production data intended to support CTR filing, set source to "live" explicitly.

5.3 Event Kinds and Field Reference

Ten event kinds are accepted, given as the kind field (or the aliases event_type / eventType). An unrecognised kind is refused at intake rather than stored silently — this is a closed set by design.

Fields common to every event kind

Canonical field	Accepted payload keys	Required?	Notes
event_id	event_id, eventId, cbsRef	Yes	Your system's own event id — this is the dedupe key.
kind	kind, event_type, eventType	Yes	One of the 10 kinds below.
ts	ts, timestamp, eventDate	Yes	ISO-8601 with timezone offset — see §3.3. Naive timestamps are rejected.
account	account, loanAccount, accountNumber	Yes	The borrowing/loan account the event is about.
amount	amount, amount_paise, txnAmount	Yes, except for rm_observation and account_status (which carry no money)	Integer paise or a decimal rupee string — see §3.3.
counterparty_account	counterparty_account, payeeAccount, beneficiaryAccount	No	Where the money went or came from, when the event has a second side.
counterparty_name	counterparty_name, payeeName	No	Truncated to 200 characters.
(anything else)	any other key you send	—	Captured verbatim in an attributes object and preserved — nothing you send is silently dropped, even if not individually validated.

Event kinds, what each unblocks, and kind-specific required fields

kind	What it is	Unblocks	Extra required fields
loan_disbursement	Funds released against a sanctioned facility	CBS-01, CBS-02	— (common fields only)
loan_utilisation	Where disbursed funds subsequently went	CBS-01	within_sanctioned_purpose (true/false/unset) recommended — optional, tri-state

kind	What it is	Unblocks	Extra required fields
loan_repayment	A repayment, with the source of the money	BEH-02	funding_source — one of own_deposits, own_bank_credit, external_bank, cash, unknown. Soft-required: if omitted, defaults to unknown, which will not clear BEH-02 — always send the real value.
cash_withdrawal	Cash taken from a loan/borrowal account	CBS-02	— (common fields only)
sale_proceeds	Proceeds of a financed asset, and whether they reached the lender	BEH-03	routed_through_lender (true/false) — hard-required. The event is rejected without it.
account_status	Opened, closed, classified, or restructured	—	No amount required. Details go in attributes.
rm_observation	A relationship manager's qualitative observation	—	No amount required. Free-text observation goes in attributes.
cash_transaction	A cash deposit or withdrawal on any account (not scoped to loan accounts)	BR-508 (CTR)	direction — must be deposit or withdrawal. Hard-required; the event is rejected without a valid value.
loan_application	A credit application, before sanction	LOS-01, LOS-03	Applicant identity and declared income — sent as free attributes (no fixed schema beyond the common fields)
collateral_valuation	A valuer's assessment of collateral offered against an application	LOS-02	asset_type and valuer_id — both hard-required. The event is rejected if either is missing; a valuation with no valuer attributed can never later be traced back to who produced it.

loan_application and collateral_valuation score differently

The other eight kinds are evaluated per payment transaction — Fraud360 looks up the loan context for whichever account a transaction in the current batch touches. An application has no payment transaction to attach to; it may predate the account's first transaction, or the account may never transact again if the application is refused. loan_application and collateral_valuation are therefore evaluated in their own pass over pending events, not alongside payment scoring. This has no effect on how you send them — same endpoint, same batch shape — only on when a resulting alert appears.

5.4 Worked Example — a Mixed Batch

A representative nightly batch covering four of the ten kinds. Every payload key shown is one Fraud360 recognises; see §5.3 for the full alias list per field.

REQUEST

```
POST https://api.fraud360.examplebank.co.in/api/ingest/b3b90261-9663-46c3-b31f-e8afc4a65442/cbs-events
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6I...
```

```
{
  "source": "live",
  "events": [
    { "payload": {
      "kind": "loan_repayment",
      "eventId": "CBS-EVT-2026-0817-88301", "eventDate": "2026-08-
17T02:10:00+05:30",
      "loanAccount": "LN-77120-9981", "amount": "45000.00",
      "fundingSource": "external_bank" } },
    { "payload": {
      "kind": "cash_transaction",
      "eventId": "CBS-EVT-2026-0817-88302", "eventDate": "2026-08-
17T11:42:00+05:30",
      "accountNumber": "SB-40011-2290", "amount": "195000.00",
      "direction": "deposit" } },
    { "payload": {
      "kind": "sale_proceeds",
      "eventId": "CBS-EVT-2026-0817-88303", "eventDate": "2026-08-
16T16:05:00+05:30",
      "loanAccount": "LN-55210-1187", "amount": "820000.00",
      "routedThroughLender": false } },
    { "payload": {
      "kind": "collateral_valuation",
      "eventId": "LOS-EVT-2026-0817-40021", "eventDate": "2026-08-
15T10:00:00+05:30",
      "loanAccount": "APP-2026-004471", "amount": "3200000.00",
      "assetType": "residential_property", "valuerId": "VLR-0091" } }
  ]
}
```

RESPONSE — 202

```
{
  "accepted": 4,
  "duplicates": 0,
  "rejected": 0,
  "errors": [],
  "batch_id": "8e2a1f0d-4c6b-4b2e-9a17-2f5d0c9b6e33"
}
```

If, for example, the `collateral_valuation` row above had omitted `valuerId`, that row alone would be rejected and reported — the other three would still be accepted:

```
{
  "accepted": 3,
  "duplicates": 0,
  "rejected": 1,
  "errors": [
    { "index": 3, "error": "collateral_valuation requires valuer_id; without it a
      deviation can never be attributed to the valuer who
      produced it" }
  ],
  "batch_id": "..."
}
```

5.5 Coverage and Status Visibility



A feed that was configured once and then went quiet looks identical to one that was never configured, unless something surfaces which kinds are missing. Two read endpoints (human/console credential, not the machine ingest credential) expose this:

Endpoint	Returns
GET /ingest/{tenant_id}/cbs-events/coverage	Every event kind, what it unblocks, how many have been received, and the timestamp of the latest one — so a stalled feed is visible per kind, not just in aggregate.
GET /ingest/{tenant_id}/status	Overall queue depth and freshness for the tenant, across both payment and CBS/LOS intake.

6. Onboarding Checklist

Before your first production call, confirm each of the following with your Fraud360 onboarding contact:

- Machine credential issued, with the correct scope (ingest, decide, or ingest+decide — §3.2)
- Base URLs received for all endpoints you will call — identity service, API gateway, and (Lane A only) decision service (§3.1)
- IP allow-list entries submitted for every address your system will call from
- Certificate binding arranged, if your integration can present one (recommended, not mandatory)
- UAT credential exercised against §4.5 / §5.4's worked examples, with matching responses observed
- (Lane A only) Fail policy — open or closed — confirmed per rail with the bank's Fraud360 administrator, and implemented on your side for the timeout case
- (Lane A only) Shadow-mode expectations understood: action is safe to act on unconditionally; would_be is reporting-only and must never drive behaviour
- (Lane B only) Which of the 10 event kinds your system will send, and the kind-specific required fields for each (§5.3) confirmed against your extract's actual field availability
- (Lane B only) source value confirmed — "live" if this feed is to support CTR filing (BR-508), per the note in §5.2
- Error-handling logic built against error.code (Appendix B), not against error.message text
- A rollback/escalation contact agreed for the go-live window (Appendix C)

Appendix A — Lane A Payment-Rail Field Reference

Every accepted key name (aliases) for the payload object inside each transaction, for the three rails in scope for this integration. Send any one of the aliases listed for a field — Fraud360 checks them in the order shown and takes the first present, non-empty value.

A.1 UPI

Canonical field	Accepted payload keys	Format
txn_id	txn_id, upiTransactionId, npcITxnId	string — required
ts	ts, txnTimestamp, timestamp	ISO-8601 with timezone offset — required
amount_paise	amount, amountRupees, txnAmount	decimal rupees (string/number) or integer paise — required
debtor_account	debtor_account, payerAccount, payerVpa	string — required
creditor_account	creditor_account, payeeAccount, payeeVpa	string — required
channel	channel	free text — default "mobile"

A.2 IMPS

Canonical field	Accepted payload keys	Format
txn_id	txn_id, rrn, imps_rrn	string — required
ts	ts, txnDate, timestamp	ISO-8601 with timezone offset — required
amount_paise	amount, txnAmount	decimal rupees or integer paise — required
debtor_account	debtor_account, remitterAccount	string — required
creditor_account	creditor_account, beneficiaryAccount	string — required
channel	channel	free text — default "netbanking"

A.3 Card

Canonical field	Accepted payload keys	Format
txn_id	txn_id, authCode, arn	string — required
ts	ts, authTime, timestamp	ISO-8601 with timezone offset — required
amount_paise	amount, authAmount	decimal rupees or integer paise — required
debtor_account	debtor_account, cardAccount, pan_token	string — required. Send a tokenised PAN reference, never a raw card number.
creditor_account	creditor_account, merchantId, merchant_id	string — required
channel	channel	free text — default "pos"

A.4 Fields common to every rail (optional)

Canonical field	Accepted payload keys	Notes
branch	branch, branch_code	—
region	region	lower-cased
product	product, account_type	lower-cased, default "savings"
customer_segment	customer_segment, segment	lower-cased, default "retail"
device_id	device_id, device, terminal_id	—
ip_addr	ip_addr, ip	—
status	status	lower-cased, default "settled"
beneficiary_added_ts	beneficiary_added_ts	ISO-8601 with timezone offset, if your system tracks when the beneficiary was first added
direction	direction	lower-cased, default "debit"

Two accounts must differ

A payload where debtor_account and creditor_account resolve to the same value after mapping is rejected outright ("debtor and creditor accounts are identical") — this is refused at the adapter, before the row ever reaches detection.

A.5 File / SFTP Delivery (Payment Rails Only)

As an alternative to the REST batch shown in §4.7, payment-rail transactions may be delivered as a file drop. **This mechanism is not available for CBS/LOS events** — those are REST-only (§5.2).

Attribute	Detail
Formats accepted	CSV, JSON-lines (.jsonl / .ndjson), or ISO 20022 XML (pacs.008 FI-to-FI credit transfer, or pain.001 customer credit transfer initiation)
Delivery path	drop/{tenant_id}/incoming/ — filename convention e.g. UPI_20260817.csv
.done marker	SFTP writes are not atomic. Write the data file first, then write an empty companion file with the same name plus .done (e.g. UPI_20260817.csv.done). The file is only picked up once the marker is present, which prevents a half-written file from being read as a short-but-plausible batch.
Idempotency	Files are keyed on their own SHA-256 hash — resending the identical file is a no-op, safe to do on any doubt about whether a prior delivery succeeded.
Size limit	200 MB per file
Row-level quarantine	Up to 5,000 unparseable rows per file are reported individually (line number and reason) via GET /ingest/{tenant_id}/files/{file_id}/quarantine — same partial-acceptance discipline as the REST batch.
ISO 20022 validation	The message's own group header (NbOfTx, CtrlSum) is checked against its content; a mismatch fails the whole file rather than silently ingesting a truncated one. Only INR-denominated messages are accepted — a foreign-currency message is refused rather than converted, since no exchange rate is available at the edge.

File status and quarantine detail are visible via GET /ingest/{tenant_id}/files and GET /ingest/{tenant_id}/files/{file_id}/quarantine (human/console credential).

Appendix B — Error Code Reference

Every error uses the envelope in §3.5. Codes below are drawn from the actual service implementation; treat any code not listed here as unexpected and report it.

HTTP status	error.code	Applies to	Meaning
400	unsupported_grant	Auth	grant_type was not client_credentials.
401	invalid_client	Both lanes	Unknown client_id or wrong client_secret. Deliberately identical for both cases — see §3.2.
403	credential_unusable	Both lanes	Credential expired or revoked.
403	ip_not_allowed	Both lanes	Calling address not on the credential's IP allow-list.
403	certificate_mismatch	Both lanes	Presented client certificate does not match the credential's binding.
403	tenant_suspended / tenant_offboarded	Both lanes	The bank's Fraud360 service is stopped.
403	(scope refusal)	Both lanes	Credential's scope does not include the capability being called (e.g. ingest-only credential calling /decide).
409	rail_not_inline	Lane A	The rail is configured for Lane B, not real-time decisioning — see §4.6.
409	no_rail_policy	Lane A	No inline decision policy configured yet for this rail.
413	batch_too_large	Both lanes (batch)	More than 5,000 transactions/events in one request — split and resend.
422	(validation error)	Both lanes	Malformed request body (FastAPI/Pydantic validation) — the request never reached row-level processing.
429	rate_limited	Both lanes (via gateway)	Too many requests from this caller; retry after the value in the Retry-After header.
5xx	internal_error	Both lanes	An unhandled fault on the Fraud360 side. No internal detail is included in the response; retry with backoff and escalate if persistent.

Row-level errors inside a 202 batch response (errors[].error) are free-text adapter messages, not stable codes — they are written to be read by a human fixing the row, not branched on programmatically. Examples appear throughout §4 and §5.

Appendix C — Contacts and Change Control

Role	Contact
Fraud360 onboarding / integration lead	To be completed at engagement kickoff
Fraud360 production support (Lane A incidents)	To be completed at engagement kickoff — must be reachable for payment-path incidents
Fraud360 production support (Lane B / batch)	To be completed at engagement kickoff
Escalation for credential issues (rotation, lockout)	To be completed at engagement kickoff

Change control

This is a versioned document. A breaking change to any request/response contract described here (a field renamed or removed, a new hard-required field, a status-code meaning changed) will be communicated to the distribution list for this document ahead of deployment, with a version bump recorded below. Additive changes (a new optional field, a new event kind, a new error code) may not warrant advance notice but will still be reflected in the next version.

Version	Date	Change
1.0	17 August 2026	Initial issue — Lane A (UPI/IMPS/Card real-time decisioning) and Lane B (CBS/LOS batch/event feed) combined specification.
1.1	17 August 2026	Added §4.8, ISO 8583 (Preview) — a second Lane A front door for card/EFT switches that cannot originate JSON. Marked Preview, not for production integration: standard-field mapping only, dialect/private-field confirmation and PCI-DSS scope decision both pending. See §4.8.1–4.8.6.